

Dashboard base-URL / /api/v1 routing audit — Stream DA

Revision: 1 Last modified: 2026-07-10T02:34:00Z

Scope

Stream DA (T1/main): audit dashboard/ (the operator SPA) for the same class of defect just fixed in ota-manager's api-client.ts (C1) — an absolute/off-origin API base URL that a same-origin CSP connect-src 'self' would block, and/or endpoint paths that don't match the real server's /api/v1 routing (server/internal/api/server.go registers every authenticated route under `r.Group(s.cfg.APIBasePath) = /api/v1`).

Scope was strictly dashboard/ — no edits to clients/, server/, submodules/; no git commands run (conductor commits).

Facts established (§11.4.6 — no guessing)

1. Base-URL resolution

dashboard/src/api/client.ts:74-79:

```
function resolveBaseUrl(): string {
  // Runtime config injection (design §3.2). Falls back to same-
  // origin /api/v1.
  const injected = (globalThis as { __HELIX_CONFIG__?:
    { apiBaseUrl?: string } })
    .__HELIX_CONFIG__.apiBaseUrl;
  return (injected ?? "/api/v1").replace(/\/+$/, "");
}
```

The exported singleton (client.ts:352, `export const apiClient = new ApiClient();`) — the one every screen (OverviewScreen.tsx, DeploymentsScreen.tsx, GroupsScreen.tsx, FleetScreen.tsx,

AuditScreen.tsx, ArtifactUploadScreen.tsx, ...) actually imports and uses — is constructed with **no explicit baseUrl argument**, so it always goes through resolveBaseUrl().

Default (no window.__HELIX_CONFIG__.apiBaseUrl injected, the normal deployment case per index.html:6-10's documented "optional inline script" comment): **"/api/v1"** — **a relative, same-origin path**. This is the opposite of the ota-manager C1 defect (which defaulted to an *absolute* http://localhost:8080).

buildUrl() (client.ts:106-118) resolves that relative base against window.location.origin:

```
const url = new URL(this.baseUrl + path, window.location.origin);
```

so every request the SPA issues in production is same-origin — CSP connect-src 'self' compliant by construction, not by accident.

An operator MAY override the base at deploy time via window.__HELIX_CONFIG__.apiBaseUrl (documented injection point, index.html:6-10) — that override, if ever set to an absolute off-origin URL, is an *operator/ops-config* decision outside the shipped client's control, not a defect in the client code being audited here.

2. Endpoint path scheme vs. real server routing

server/internal/api/server.go:110-129:

```
r.GET("/healthz", s.handleHealthz)
r.GET("/readyz", s.handleReadyz)

v1 := r.Group(s.cfg.APIBasePath) // APIBasePath default = "/api/v1" (config.go:23)
v1.Use(apiSecurityHeadersMiddleware())

v1.POST("/auth/login", s.handleLogin)
v1.POST("/auth/refresh", s.handleRefresh)

auth := v1.Group("")
auth.Use(s.authMiddleware(), s.auditMiddleware())
{
    auth.GET("/devices/:deviceId/status", ...)
    auth.POST("/artifacts/upload", ...)
    auth.GET("/artifacts/:artifactId", ...)
    auth.POST("/releases", ...) / auth.GET("/releases", ...) /
        auth.GET("/releases/:releaseId", ...)
```

```

auth.POST("/deployments", ...) / auth.GET("/
  deployments", ...) / auth.GET("/
  deployments/:deploymentId", ...)
auth.POST("/deployments/:deploymentId/rollout", ...) /
  auth.GET(same) / auth.POST(.../evaluate)
auth.POST("/deployments/:deploymentId/recall", ...) /
  auth.GET(.../rollbacks)
auth.GET("/devices/:deviceId/telemetry", ...) / auth.GET("/
  telemetry/overview", ...)
auth.POST("/groups", ...) / auth.GET("/groups", ...) / auth.GET /
  PATCH/DELETE("/groups/:groupId", ...)
auth.GET/POST("/groups/:groupId/members", ...) /
  auth.DELETE(".../members/:deviceId")
auth.GET("/audit", ...)
...
}

```

Correction to the task’s stated premise: the premise “the server registers EVERY route under APIBasePath /api/v1, no bare routes” is not quite exact for *this* server — /healthz and /readyz are deliberately registered **unversioned**, before the v1 group (server.go:125-127, comment: “Health/ readiness are unversioned, unauthenticated operational probes.”). Stating this as fact rather than assuming the premise, per §11.4.6.

Every data-bearing endpoint the dashboard actually calls in dashboard/src/api/client.ts is written as a **bare relative path with no /api/v1 prefix** — e.g. /auth/login, /releases, /deployments, /deployments/{id}/rollout, /devices/{id}/status, /devices/{id}/telemetry, /telemetry/overview, /groups, /groups/{id}/members, /audit, /artifacts/upload, /artifacts/{id}. Because baseUrl already **is** /api/v1 (resolveBaseUrl() default) and buildUrl() does this.baseUrl + path (client.ts:108), every one of these resolves to /api/v1/<path> — **exactly matching every route listed above**. Cross-checked one-by-one against server.go’s route table: releases, deployments (+ rollout / rollout/evaluate / recall / rollbacks), devices/:id/status, devices/:id/telemetry, telemetry/overview, groups (+ members), audit, artifacts/upload, artifacts/:id, auth/login, auth/refresh — all match. No path double-prefixes or omits /api/v1.

3. The one genuine path oddity — health(), already known + already honestly documented (not a new defect)

client.ts:344-348:

```
// --- health (best-effort; not a defined MVP route – design §8)
// -----
health(): Promise<HealthStatus> {
  return this.json<HealthStatus>("/healthz", { anonymous: true });
}
```

Because `baseUrl = /api/v1`, this resolves to `/api/v1/healthz` — but the server serves `/healthz` **unversioned** (fact #2 above), so this specific call 404s against the real server. This is real, but:

- It is **already known and already honestly documented** in the source (`client.ts:344` comment) and in the one caller, `dashboard/src/screens/OverviewScreen.tsx:1-3, 12, 18-19`: > “Recent releases + best-effort server-health badge. The `/healthz` route > is not a defined MVP JSON endpoint (design §8) — the badge is best-effort > and degrades silently.” ... `{/* Best-effort health badge – silent if the > route is not served. */} {health.data ? <Badge tone="ok">...</Badge> : null}`
- `useApi (src/query/useApi.ts)` catches the rejected fetch into error state; `health.data` simply stays `null`; the UI conditionally renders nothing for the badge — no crash, no user-visible broken feature, no silent-PASS-bluff (the failure is visible as “badge absent,” not reported as success).
- This is the *opposite* shape of the C1/CSP-off-origin defect class this audit targets (wrong-prefix, not wrong-origin) and is a pre-existing, self-documented, non-blocking best-effort probe, not a newly-discovered regression this stream is chartered to fix. Per the task’s own instruction 4 (“if already correct... say so, do NOT invent a fix”) and per §11.4.6 (state facts, don’t invent unrequested fixes outside strict scope), this is reported as a fact and left as-is — fixing it would mean either adding a bare-route client method that bypasses the typed `/api/v1` base (a new special case) or changing server routing (`server/` — explicitly out of this stream’s scope). No source change made for this item.

Verdict

ALREADY-CORRECT. The dashboard’s `ApiClient` does not have the ota-manager C1 defect:

- Default base URL is **relative `"/api/v1"`**, resolved same-origin via `window.location.origin` — CSP `connect-src 'self'` safe, not off-origin.

- Every real data-endpoint path is written bare (no `/api/v1` prefix in the path string) *because* the base already carries that prefix — verified 1:1 against the server’s actual route table in `server/internal/api/server.go`. No bare-vs-prefixed mismatch of the ota-manager kind exists.
- The one path that resolves incorrectly (`health()` → `/api/v1/healthz` instead of the server’s unversioned `/healthz`) is a pre-existing, self-documented, best-effort, silently-degrading probe — not the off-origin/CSP-blocking defect class this audit targets, and not touched per the “do not invent a fix” instruction.

No source fix was required or made for the base-URL / path-prefix defect class. One **additive** evidence-closing test was added (see below) because every pre-existing test in `client.test.ts` constructed the client with an **explicit** absolute test base (`new ApiClient("http://api.test/api/v1")`), so none of them had ever actually exercised `resolveBaseUrl()` or the production `apiClient` singleton’s default. That was a real coverage gap for proving the *shipped* default is safe (§11.4.123 rock-solid-proof mandate) — closing it is evidence, not a fix.

Test added (additive evidence, not a RED → GREEN bug fix)

`dashboard/src/api/client.test.ts` — new `describe` block: “default `apiClient` singleton — same-origin relative `/api/v1` base (audit evidence, not a fix)”. Imports the real exported `apiClient` singleton (previously unused in this test file) and asserts:

1. `requested.origin === window.location.origin` (same-origin, not an absolute off-origin host).
2. `requested.pathname === "/api/v1/releases"` (correctly prefixed against the real server’s route group).
3. `!calls[0].url.startsWith("http://localhost:8080")` (explicitly rules out the ota-manager C1 pattern).

Falsifiability proof (§11.4.115-style polarity check on the guard itself)

The guard is not a bluff — proven by temporarily mutating `resolveBaseUrl()`’s fallback from `"/api/v1"` to the ota-manager C1 pattern `"http://localhost:8080"` and re-running the single new test:

```
RED (mutated to "http://localhost:8080"):  
  ✗ resolves the shipped singleton's default base to same-origin  
+ /api/v1, never an absolute off-origin host  
  AssertionError: expected 'http://localhost:8080' to be  
  'http://localhost:3000'  
  Tests 1 failed | 12 passed (13)
```

```
GREEN (reverted to the real "/api/v1"):  
  ✓ src/api/client.test.ts (13 tests) 12ms  
  Test Files 1 passed (1)  
  Tests 13 passed (13)
```

Mutation was applied and reverted in-place on `client.ts` only for this proof (no other change survives); the file's final committed state is byte-identical to before the mutation except for the one new test block in `client.test.ts`.

Full-suite results (post-verification)

```
$ npx tsc --noEmit -p tsconfig.json  
(clean, exit 0)
```

```
$ npx vitest run  
Test Files 12 passed (12)  
Tests 107 passed (107) <- before the new test was added
```

After adding the new test:

```
$ npx tsc --noEmit -p tsconfig.json  
(clean, exit 0)
```

```
$ npx vitest run src/api/client.test.ts  
Test Files 1 passed (1)  
Tests 13 passed (13) <- was 12; +1 new evidence test
```

No dist artifacts were rebuilt (dashboard/dist/ untouched per instruction).

Files touched (dashboard/ only, per strict scope)

- dashboard/src/api/client.test.ts — added one additive describe block (13 tests total, was 12). No production source (client.ts or any screen) was modified in the final state.

Bottom line

The dashboard does **not** have the ota-manager C1 defect. Base URL defaults to relative, same-origin /api/v1; every real endpoint path resolves correctly against the server's actual /api/v1-grouped routes. The only path oddity (health() → /api/v1/healthz vs. the server's unversioned /healthz) is a pre-existing, already-documented, silently-degrading best-effort probe outside the audited defect class, left untouched per instruction. One additive test was added purely to close an evidence gap (no prior test exercised the real default base), proven falsifiable by a temporary mutate-and-revert cycle.